

LAPORAN WORKSHOP DevOps
Bab 2 - Konsep Container dan Instalasi Docker



Nama : Fitria Sari Ayuningtyas

NRP : 3126640013

Dosen Pengajar : Dr. Ferry Astika Saputra, ST, M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)
TAHUN 2026

A. Verifikasi dan Hasil Pengujian

Verifikasi dilakukan untuk memastikan Docker dapat menjalankan container dan service dengan baik. Pengujian dimulai dengan menjalankan container hello-world, kemudian dilanjutkan dengan menjalankan Nginx menggunakan image nginx:1.26. Container Nginx berhasil berjalan dan dapat diakses melalui localhost:8080. Selanjutnya dibuat custom image pens-web:1.0 menggunakan Dockerfile dan file index.html. Image tersebut berhasil dijalankan sebagai container pens-app dan dapat diakses melalui localhost:9090. Hasil curl -v menunjukkan status HTTP/1.1 200 OK serta menampilkan halaman Docker Lab PENS. Bukti pengujian ditunjukkan melalui hasil docker ps, akses Nginx, proses build custom image, dan hasil akses custom web.

Bukti pengujian yang dilampirkan:

1. Hasil docker ps yang menunjukkan container Nginx berjalan.

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-2$ docker run -d --name web-public -p 8080:80 nginx:1.26
121e73a6aa5f65d9b68888a3a18e2c9e10a297f755dadb1ddf44c007c199ae3f
```

2. Hasil curl http://localhost:8080 yang menunjukkan Nginx dapat diakses.

```
asus@LAPTOP-OC46B530:~/docker-lab/bab-2$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color:scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

3. Hasil proses build pens-web:1.0.

```
asus@LAPTOP-OC46B530:~/docker-lab/custom-web$ docker build -t pens-web:1.0 .
[+] Building 10.1s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 102B
=> [internal] load metadata for docker.io/library/nginx:1.26-alpine
=> [internal] load build context
=> resolving docker.io/library/nginx:1.26-alpine
=> [1/2] FROM docker.io/library/nginx:1.26-alpine
=> resolving docker.io/library/nginx:1.26-alpine
=> sha256:d987ab110a5badaef9d77115759f91e8a5d0bea70045f4de7a50959be1b04c 404B / 404B
=> sha256:a7c61bccdef1ba5e7ee3b4f8a8d77754a6e01a598271b70dd220d8041156b 1.40KB / 1.40KB
=> sha256:5a0872a0c64f7351f9c91c3e834e31bcac00d1a124ef3a0a070c210f718d 15.19KB / 15.19KB
=> sha256:b065133baaa3f10ace1ba6d5cfc18e270ba40bca95328635f0ba3f13855 956B / 956B
=> sha256:c35c2aa19c882e0ca054153d04a4b58904f72807055c441be84f27401714f25d 1.21KB / 1.21KB
=> sha256:5e18ef93c43509cd1c48d1ae900cf7e92ee1ae11be6b3bdefc489f4be8efad9 628B / 628B
=> sha256:91408eaf00479e4b1710102ed1e2400e92231d11c32276f024867f5f39e97b 1.76KB / 1.76KB
=> sha256:b0a6a4f000f05eb27e479bd08789a29e52769b21bcbcd01481eae0b043dc 1.63KB / 1.63KB
=> extracting sha256:0a0a5f000f05eb27e479bd08789a29e52769b21bcbcd01481eae0b043dc
=> extracting sha256:91408eaf00479e4b1710102ed1e2400e92231d11c32276f024867f5f39e97b
=> extracting sha256:5e18ef93c43509cd1c48d1ae900cf7e92ee1ae11be6b3bdefc489f4be8efad9
=> extracting sha256:b065133baaa3f10ace1ba6d5cfc18e270ba40bca95328635f0ba3f13855
=> extracting sha256:d987ab110a5badaef9d77115759f91e8a5d0bea70045f4de7a50959be1b04c
=> extracting sha256:c35c2aa19c882e0ca054153d04a4b58904f72807055c441be84f27401714f25d
=> extracting sha256:a7c61bccdef1ba5e7ee3b4f8a8d77754a6e01a598271b70dd220d8041156b
=> extracting sha256:5a0872a0c64f7351f9c91c3e834e31bcac00d1a124ef3a0a070c210f718d
=> [internal] load build context
=> transferring context: 90B
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> exporting layers
=> exporting manifest sha256:f608205481b5df06823471d0ef140775a74623a009cb4cf1bd7f0dc53437
=> exporting config sha256:4e0ad4f06f7000f72b1e072624094910a2e502e6951a0be9f2ba2d461e09285
=> exporting attestation manifest sha256:d42a05fd4842e08010258dc3302f5650e26378b09781ba195ae5bc1c4310
=> exporting manifest list sha256:72c0a55c8995c75be0063490cadf218f22e7c53298510ac7a68fc7c3550608
=> naming to docker.io/library/pens-web:1.0
=> unpacking to docker.io/library/pens-web:1.0
```

4. Hasil curl -v http://localhost:9090 yang menunjukkan status 200 OK dan halaman custom.

```
asus@LAPTOP-OC46B530:~/docker-lab/custom-web$ curl http://localhost:9090
<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>
```

```

asus@LAPTOP-0C46B530:~/docker-lab/custom-web$ curl -v http://localhost:9090
* Host localhost:9090 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:9090...
* Established connection to localhost (::1 port 9090) from ::1 port 36460
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:9090
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.26.3
< Date: Fri, 04 Sep 2026 09:37:16 GMT
< Content-Type: text/html
< Content-Length: 61
< Last-Modified: Fri, 04 Sep 2026 09:20:58 GMT
< Connection: keep-alive
< ETag: "6a9a8d7a-3d"
< Accept-Ranges: bytes
<
<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>
* Connection #0 to host localhost:9090 left intact

```

B. Troubleshooting dan Analisis Hasil

Selama praktikum terdapat pesan Unable to find image 'ubuntu:22.04' locally ketika menjalankan container Ubuntu. Pesan tersebut tidak menunjukkan kegagalan, tetapi menandakan bahwa image ubuntu:22.04 belum tersedia secara lokal. Docker kemudian secara otomatis mengambil image dari Docker Hub dan melanjutkan proses pembuatan container.

Diagnosis dilakukan dengan memperhatikan output proses pull image dan perubahan prompt menjadi root@...:/#, yang menunjukkan bahwa container berhasil dijalankan secara interaktif. Verifikasi lebih lanjut dilakukan menggunakan cat /etc/os-release dan diperoleh informasi Ubuntu 22.04.5 LTS.

Selain itu, validasi service dilakukan menggunakan docker ps, docker logs, dan curl. Pemeriksaan tersebut menunjukkan bahwa container Nginx berada dalam status berjalan, log tidak menunjukkan kesalahan startup, dan request HTTP mendapatkan respons 200 OK. Pemeriksaan docker network ls juga menunjukkan adanya network default Docker seperti bridge, host, dan none.

C. Analisis Keamanan dan Operasional

Penggunaan container mempermudah proses menjalankan aplikasi karena lingkungan aplikasi dapat dikemas dalam sebuah image yang konsisten. Namun, terdapat beberapa risiko keamanan dan operasional yang perlu diperhatikan. Image yang digunakan sebaiknya berasal dari sumber yang terpercaya karena image dapat menjadi bagian dari rantai pasok perangkat lunak. Selain itu, penggunaan tag latest kurang baik untuk deployment yang membutuhkan hasil yang reproducible karena versi image dapat berubah.

Publikasi port juga perlu diperhatikan. Port yang tidak diperlukan sebaiknya tidak dibuka ke host atau jaringan eksternal. Akses terhadap Docker daemon juga harus dibatasi karena pengguna yang memiliki akses terhadap Docker dapat memperoleh hak yang sangat tinggi pada sistem host.

D. Rekomendasi Production-like Environment

Jika konfigurasi praktikum diterapkan pada lingkungan production-like, image sebaiknya menggunakan versi yang jelas dan diverifikasi, serta dilakukan vulnerability scanning sebelum digunakan. Port yang dipublikasikan perlu dibatasi sesuai kebutuhan aplikasi. Konfigurasi Docker juga sebaiknya dikelola secara konsisten agar lingkungan pengembangan dan deployment memiliki konfigurasi yang sama.

Selain itu, container perlu dipantau menggunakan mekanisme monitoring dan logging. Untuk aplikasi yang menyimpan data, penggunaan volume dan mekanisme backup juga perlu direncanakan agar data tidak hilang ketika container dihentikan atau dibuat ulang.

E. Evaluasi dan Latihan Mandiri

1. Mengapa penggunaan tag latest tidak dianjurkan untuk deployment yang harus reproducible?

Tag `latest` dapat menunjuk ke versi image yang berbeda ketika image diperbarui. Hal ini dapat menyebabkan hasil deployment berubah meskipun konfigurasi yang digunakan sama. Penggunaan versi tertentu membuat deployment lebih konsisten dan mudah ditelusuri.

2. Jelaskan peran containerd dan runc dalam arsitektur Docker.

`containerd` berperan sebagai container runtime tingkat tinggi yang menangani lifecycle container, image, dan pengelolaan container. `runc` merupakan runtime tingkat rendah yang menjalankan container berdasarkan standar OCI dan berinteraksi dengan fitur isolasi Linux.

3. Apa konsekuensi keamanan dari memasukkan user ke group docker?

User yang memiliki akses ke Docker daemon dapat melakukan operasi dengan hak istimewa yang tinggi terhadap sistem host. Oleh karena itu, akses ke Docker perlu diberikan hanya kepada user yang memang membutuhkannya.

4. Bandingkan layer image `nginx:1.26-alpine` dan image custom yang dibuat.

Image `nginx:1.26-alpine` menjadi base image yang digunakan dalam Dockerfile. Image custom `pens-web:1.0` menggunakan base tersebut kemudian menambahkan layer untuk file `index.html` dan konfigurasi yang didefinisikan dalam Dockerfile. Dengan konsep layer, bagian yang sama dari base image dapat digunakan kembali.

5. Kapan sebaiknya memilih VM daripada container?

VM lebih sesuai ketika diperlukan isolasi sistem operasi yang lebih lengkap, kernel yang berbeda, atau kebutuhan lingkungan yang tidak cocok dijalankan dalam container. Container lebih sesuai untuk aplikasi yang membutuhkan deployment cepat dan lingkungan yang ringan.

F. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, Docker berhasil digunakan untuk menjalankan beberapa container, mulai dari container `hello-world`, Nginx, hingga container Ubuntu. Pengujian Nginx menunjukkan bahwa container dapat berjalan dan diakses melalui port host. Selain itu, custom image `pens-web:1.0` berhasil dibuat menggunakan Dockerfile dan dapat menjalankan halaman web sederhana.

Praktikum ini menunjukkan bahwa container dapat digunakan untuk menyediakan lingkungan aplikasi yang lebih ringan dan konsisten. Namun, penerapan container pada lingkungan production-like tetap perlu memperhatikan keamanan image, pengelolaan akses, publikasi port, versioning image, serta monitoring dan pengelolaan data.